Machine Learning: License plate recognition

Author

Gianfranco Demarco

Introduction

License plate recognition is the problem of transcribing the license plate of a vehicle starting from an image or a video. A good part of this work consisted of identifying and building a dataset suitable for the problem. A Deep Learning pipeline is then explored, proposing also enseble learning techniques to improve the reliability of the process. Also, there has been focus on the evaluation of the proposed solution and on the operationalizing of the system. The proposed solution represents a complete pipeline, with great space for improvements in many different areas.

License plate recognition

License plate recognition (LPR) is the problem of transcribing the license plate of a vehicle starting from an image or a video. Systems that can perform LPR are useful for different tasks, from traffic surveillance to roadside assistance. Even if it is a very studied problem, most of the solution found on the web are related to propietary system of private companies, which offer paid services. Also, suitable dataset are very scarce, given that there is lack of good quality images and annotations.

The problem of license plate recognition is often split up in serveral smaller problems, involving different machine learning models [9]:

1. Identify the area which contains the plate
2. Perform segmentation of the individual characters
3. Perform Optical character recognition (OCR) on the segmented areas

In this work, the second second step was omitted, and so the steps which are carried out are:

1. Identify the area which contains the plate
2. Perform OCR on whole identified area

This choice has been made to reduce the number of the machine learning models involved by 3 to 2, which reduces the possibility of errors and also makes the training process more affordable. Obviously this makes the work of the OCR model more difficult, but given that there are big pre-trained OCR models available, this option was preferred.

The type of models chosen from this work are aligned with the current trends in the machine learning research community, so Deep Learning models involving CNNs were used.

Dataset

The first part of this work consisted in an exploratory research of suitable datasets, which was disappointing. There is a number of datasets involving car images [10], but very few are focused on the plates; moreover, most of them don't include the bounding box labels referring to the plates [4] or the transcription of the plates; lastly, for some dataset, when the images are rescaled to a size viable for training (e.g. 256x256 pixels) the plates and their details become unrecognizable [8]. The few datasets which seem to be complete [2], require a formal request to the owners to be used.

The chosen approach was to build a new dataset, starting from the existing datasets, using a human-in-the-loop process.

Human-in-the-loop

The chosen datasets were Kaggle's Car Plate Detection dataset [6] and the License Plate Detection, Recognition and Automated Storage dataset. [4]

The first contains car images and annotations about the plates bounding box. The bounding box is represented as four numbers: min_x, min_y, max_x, max_y; this is a standard in bounding box representation, as these numbers are enough to draw a rectangle around the point of interest. A License Plate Detection model, which has the objective of finding the license plate in an image, has to be a regressor predicting these 4 points.

The dataset also contains images which are not good to train a model meant to be used with images taken in real life situations. This means that this dataset is not suitable to be directly used to train a production-ready model.

The latter dataset contains pictures of cars all taken from the same prospect and style (light, angle etc.).

The process designed to produce a novel annotated dataset, starting from these 2 datasets, with a human-in-the-loop approach is the following:

1. Train a License Plate Detection model using the first dataset
2. Use the model to predict the plate position on the second dataset
3. Show the predictions on the images. If the prediction is good (*judged by a human*), save it and move the corresponding image and the generated label to the training set
4. Repeat

This process allowed to create a new dataset, composed of 433 clean and clear images. Then, for these images, the license plate was transcripted by hand.

From a posterior analysis, this process resulted to be probably slower and less precise than if a tagging software was used. However, this is probably due to the fact that the first dataset, used for training, wasn't of a very good quality, and it was also foundamently different from the second dataset. In a situation were a good License Plate Detection model is already in place, this can be an effective way of generating new training data.

Data augmentation

The final dataset still was formed by only 433 images. To improve the dimensionality of the dataset and the reliability of the trained models, data augmentation was applied. In particular, these transformation were applied to each of the images, obtaining 3 new images:

- blur the image with a gaussian filter
- increase the brightness of each pixel by 30
- decrease the brightness of each pixel by 30

With this data augmentation, the dataset passed from 433 images to 1732 images. This transformations were chosen because they are position, rotation and scale invariant transformations, and this means that the license plate bounding box annotation doesn't have to be recomputed. However, further transformation can be applied to increase even further the size of the dataset. The training process did benefit from this increase in the size of the dataset, as it was less prone to overfitting.

Preprocessing

All of the images used in this work were scaled down to a size of 256x256 pixels, to reduce the computational requirements of the training phases. This required to scale also the annotations related to those images. For some images, this rescaling made it hard to read the plate even for a human observer; also, since not all of the original images were in a 1:1 aspect ratio, some images were distorted. The annotations coming along the images were also preprocessed, transforming them from a complicated XML format to a plain CSV format.

Experiments

To create the License Plate Detection models, primarly CNNs were used. Many different architecture were explored and evaluated, both using transfer learning with pre-trained model and starting from scratch. For each experiment:

- 20% of the data was kept out as test data
- of the remaining 80%, 80% was used as training data and the other 20% was used as validation data
- the models were trained on the training data, and metrics about validation data were computed during the training phase
- when the training phase converged, the models were trained again on the first 80% split and evaluated against the 20% test set.
- finally, the models were trained again using all of the data to be used in production

The loss function optimized by the training process is the *Root Mean Squared Error*, using the *Adam* optimizer.

The RMSE is defined as:

$$RMSE = \sqrt{\sum_{i=1}^{n}(prediction - ground_truth)^2/n}$$

where, in this scenario, n = 4 = the values predicted by the model.

Fine-tuning for Plate Detection

At first, a fine tuning approach was explored. Using Keras as machine learning library, many different pre-trained models [5], trained on the ImageNet dataset [3], are available. These model vary in generation (the time when they were firstly introduced), size (number of parameters and physical size of the model's file) and inference time. In particular, the models *MobileNet* and *EfficientNet* were used. The fine tuning process consists of downloading a model which have already been trained on a huge dataset, like those mentioned above, getting rid of the last layer, attach an output layer compliant with the task at hand and restart the training process. It is important to freeze the layers that were downloaded, in order to not corrupt the training that they already had. Also, between the downloaded layers and the output layers, more layers can be arbitrarily added. This process usually leads to big models, with millions of frozen parameters (cannot be trained/changed) and a lot less trainable parameters.

Custom models for Plate Detection

After experimenting with models built from scratch, it seemed clear that the performance obtained with this method were very similar to those obtained with transfer learning. Because it created models which were much smaller (9MBs vs. hundreads of MBs), this was the method that was chosen for the final system.

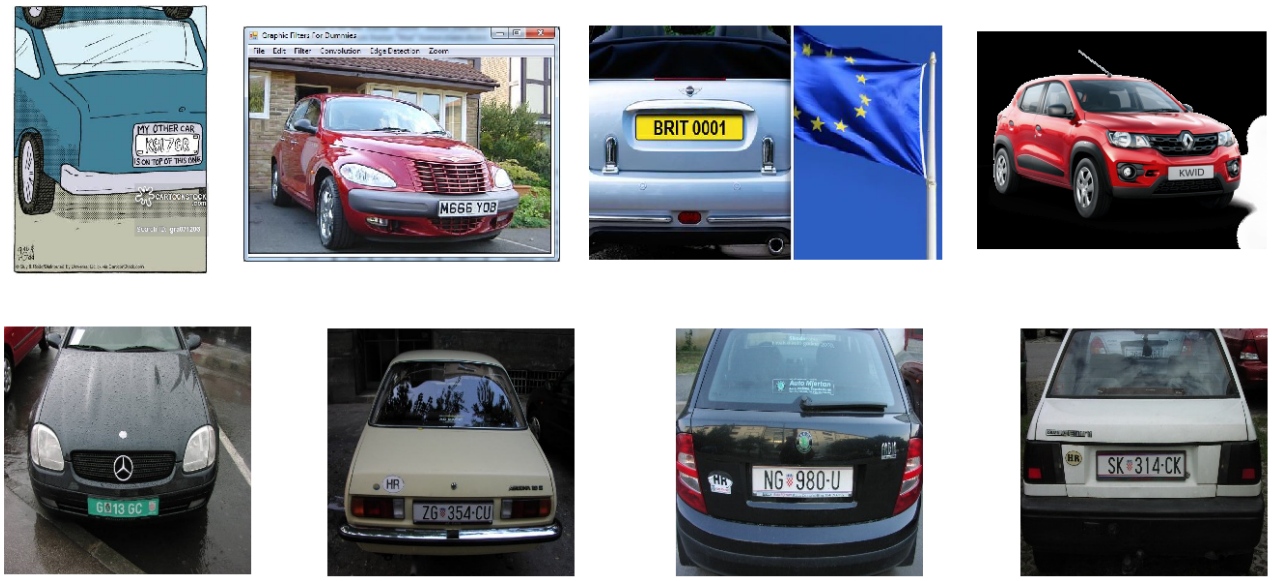

Figure 1: *Top row: some of the images of the original dataset, which are not good to be used with real world images. Bottom row: a sample of the images of the target dataset*

OCR model

The OCR is the model which takes as input the license plate, cropped from the original image according to the output of the License Plate Detection model, and gives as output the characters that it recognizes in the given image. For this work, a pre-trained OCR model provided by HuggingFace [1] was employed. This model is the "Microsoft Trocr Printed" in its large version [7]. No fine-tuning was performed on this model.

Final Pipeline

The final pipeline, performing the task of License Plate Recognition, is composed by pre-processing steps, different models applied to the data, and post-processing steps. The flow is the following:

1. Pre-process the input image
2. Apply different models to the image to perform License Plate Detection
3. For each of the predicted bounding box, perform OCR
4. Post-process the results

Step 1: Pre-processing

The pre-processing phase consists of resizing the input image to the size of 256x256 pixels, which is the size accepted by the License Plate Detection models.

Step 2: License Plate Detection

In order to increase the robustness of this step, instead of training and using just one model to perform License Plate Detection, three models are used in parallel:

- A model trained on the original image
- A model trained on a black and white version of the image
- A model trained on the edge-detection result of the image

Obviously, before being fed into the second and third model, the same black and white transformation and edge detection used in the training phase are rispectively applied to the image.

These 3 models give 3 different candidate areas for where the license plate is.

Moreover, this step is expansible to any number of models.

Step 3: OCR

OCR is then performed on each of the areas described by the bounding boxes predicted on the previous step. In this case, 3 OCR will be conducted on the 3 areas predicted. This will give 3 (possibly) different license plate predictions.

Step 4: Postprocessing

The OCR model isn't capable of detecting if in the image fed into it actually appears any text. This means that even when clearly there is no text, it will produce a sequence of characters as results. These cases are usually easy to intercept, because the OCR model will produce very short sequences (maybe composed just by a couple of special characters) or very long sequences. Also, because the pipeline produces 3 different predictions, a majority vote is applied at the end.

Figure 2: *Transofrmations applied to augment the images in the dataset. From left to right: the original image; the image obtained after applying a blurry filter; the image obtained after increasing the brightness of each pixel by 30; the image obtained after decreasing the brightness of each pixel by 30*

The post-processing steps are:

1. Clean every prediction by any special character. Even if the OCR can detect special characters and blank spaces, they don't have any semantic meaning so it is better to remove them.
2. Discard plates which are too short (< 4 characters) or too long (> 12 characters)
3. Apply a majority vote system to the predictions. If there is a winner, that is the output of the pipeline. If there are more winners, all of them are returned. It is then responsibility of the application using this system to choose how to manage that situation.

The post-processing step, other than improving the reliability and precision of the system, enables a very powerful feature, which is the possibility of returning a KO prediction. If none of the predictions pass through the post-processing step, then the License Plate Recognition can be said to be inconclusive. This would be very useful in automated systems, or to give instruction to a human user to submit a new image.

Evaluation

Evaluation is an important phase in every machine learning system. In this system, two different types of models are used, so two different evaluation are conducted. *RMSE* and *IOU* are used to estimate the goodness of the License Plate Detection models; *Accuracy* and *Levenshtein Distance* are used to estimate the goodness of the OCR model. Comparing the values of *Accuracy* and *Levenshtein Distance*, calculated first on the ground truth bounding boxes and then on the predicted bounding boxes, allows to give an estimate for the overall performance of the system.

RMSE

The *Root Mean Squared Error* indicates how on average each prediction is far away by the ground truth values. The RMSE is defined as:

$$RMSE = \sqrt{\sum_{i=1}^{n}(prediction - ground_truth)^2/n}$$

For the Licence Plate Prediction models, the *RMSE* is then used both as the objective of the optimization and as an evaluation metric.

Even if the *RMSE* is a common used metric, it is not perfect for this use-case. Imagine two different bounding box predictions having the same value for *RMSE*, but in one case the resulting bounding box is smaller and included into the the ground truth bounding box, in the other it is bigger and includes the bounding box. Even if the values of the *RMSE* are the same, the bigger predicted bounding box is preferable because it will contain all of our Region of Interest (plus some surrounding area), and it will give a chance to the OCR model to give a correct prediction. This will be more difficult if the plate is truncated by the plate recognition model.

IOU

For object detection problems, a widely used metric is *Interseciton Over Union*. [11] It measures how good is the predicted area, taking the ratio between

- the *intersection* of the predicted area and the ground truth
- the *union* of the predicted area and the ground truth

This means that the IOU metric accounts not only for how much of the region of interest is captured into the prediction, but also how much of uninteresting areas is contained as well. The IOU value ranges from 0 (the prediction and the ground truth are disjoint) to 1 (the prediction corresponds exactly to the ground truth).

Accuracy and Levenshtein Distance

Accuracy is the measure of how many of the predictions are exactly equal to the ground truth. In this work, it is applied to the plates transcription. This is probably the best measure to estimate the performance of this system. However, a character-based measure of the predictions, indicating the distance between the predicted text and the ground truth can also be computed. This

Figure 3: *Example of bounding box predictions with their IOU values. The red boxes are the ground truth, the green boxes are the predictions.*

can be a useful indication to further improve the OCR component of the system. For this reason, the *Levenshtein Distance* is employed.

Results

The metrics for the License Plate Detection models are similar to each other. The predictions are on average 12 pixels off the ground truth coordinates. The IOUs score on average 0.60, meaning that the models are far from perfect. However, given that the results are *ensembled* down in the pipeline, these models could still provide good enough predictions.

Model	RMSE	IOU
Raw images	12.66	0.5885
Black & White images	11.32	0.5980
Edge detection images	12.45	0.5921

Table 1: *Metrics for the License Plate Detection models*

To evaluate the OCR model alone, it has been tested on the ground truth bounding boxes of the plates. Here it scored an accuracy of 72%. However, this model can be fine-tuned for better performances.

Model	Accuracy	Lev Distance
Ground thruth	72.26%	0.51

Table 2: *Metrics for the OCR model. These are calculated on the ground truth bounding boxes, e.g. as if the License Plate Detection models were perfect*

The accuracy of the OCR drops to an average of 54% when evaluated on the predictions of the License Plate Detection models, and the Levenshtein Distance raises to an average of 1.4 from 0.51. This is due to the fact that the error from the two models (License Plate Detection and OCR) sum when they are used in cascade.

Moreover, these metrics are computed for each model alone and do not take advantage of the majority vote used by the final system.

Operationalizing the models

An aspect that is often ignored when working on machine learning prototypes is the *operationalization* of the system, which comprehends all of the operations needed to put the system in production. To complete the system proposed with this work, the whole pipeline has been made accessible by end users. The backend is composed by a web server, written in Python and leveraging on the Flask library. This server exposes an endpoint which receives an image and returns the predicted license plate after running the whole pipeline as described in this report. The frontend chosen is a Telegram Bot. The users can send the image in the chat with the bot and receive the predictions directly into their conversation.

Model	Accuracy	Lev Distance
Raw Images	54.02%	1.36
Black & White images	52.01%	1.52
Edge detection	56.00%	1.4

Table 3: *Metrics for the OCR model, computed on the bounding boxes predicted by all of the License Plate Detection models*

Future works and improvements

This work brought to the design and implementation of a complete pipeline regarding the problem of License Plate Recognition. It starts with the building of the dataset, shows how to implement and ensemble different models, and focuses also on the evaluation and operationalization of the system as a whole. However, many improvements can be made in different areas of the system to make it ready to face up a real-life deployment.

- Most of the focus should go on the dataset. Better and bigger models cannot achieve much if the data isn't enough and of good quality. A whole new dataset was created with this work, but much can still be done. The final dataset contains images which are all similar, with the same light and with the plate in the same position. Improving the number and the variety of images would play a big role in the improvement of the system
- The data augmentation technique can be further expanded to help improving the dataset, implementing also transformations that alter the position of the plate
- Fine tuning the OCR model, using the plates that have been manually labelled during this work, can help the model to tune on the styles of characters present on the plates
- More effort can be put in post processing. For example, there are regex that can be applied to license plates to check if they are valid. These are country specific. They can be applied to have a double benefit:
  1. if the plate pass the regex check, the system can also recognize the corresponding country
  2. applyng the regex check would heavily reduce the number of false positives

References

[1] Hugging Face. "Hugging Face". In: 9 (). URL: https://huggingface.co/.

[2] Laboratório Visão Robótica e Imagem Laboratório Visão Robótica e Imagem. "UFPR-ALPR Dataset License Agreement". In: 5 (). URL: https://web.inf.ufpr.br/vri/databases/ufpr-alpr/license-agreement/.

[3] ImageNet. "ImageNet". In: 8 (). URL: https://www.image-net.org/.

[4] Zoran Kalafatić. "License Plate Detection, Recognition and Automated Storage". In: 3 (). URL: http://www.zemris.fer.hr/projects/LicensePlates/english/results.shtml.

[5] Keras. "Keras pretrained models". In: 7 (). URL: https://keras.io/api/applications/.

[6] Larxel. "Car Plate Detection". In: 6 (). URL: https://www.kaggle.com/datasets/andrewmvd/car-plate-detection?resource=download.

[7] Minghao Li et al. *TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models*. 2021. arXiv: 2109.10282 [cs.CL].

[8] Robert Lucian. "License Plate Dataset". In: 4 (). URL: https://github.com/RobertLucian/license-plate-dataset.

[9] Xin Wang Michael Yang. "Recognizing License Plates in Real-Time". In: 1 (). URL: https://arxiv.org/abs/1906.04376.

[10] Roboflow. "License plates US-EU". In: 2 (). URL: https://public.roboflow.com/object-detection/license-plates-us-eu.

[11] Adrian Rosebrock. "Intersection over Union (IoU) for object detection". In: 11 (). URL: https://pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/.